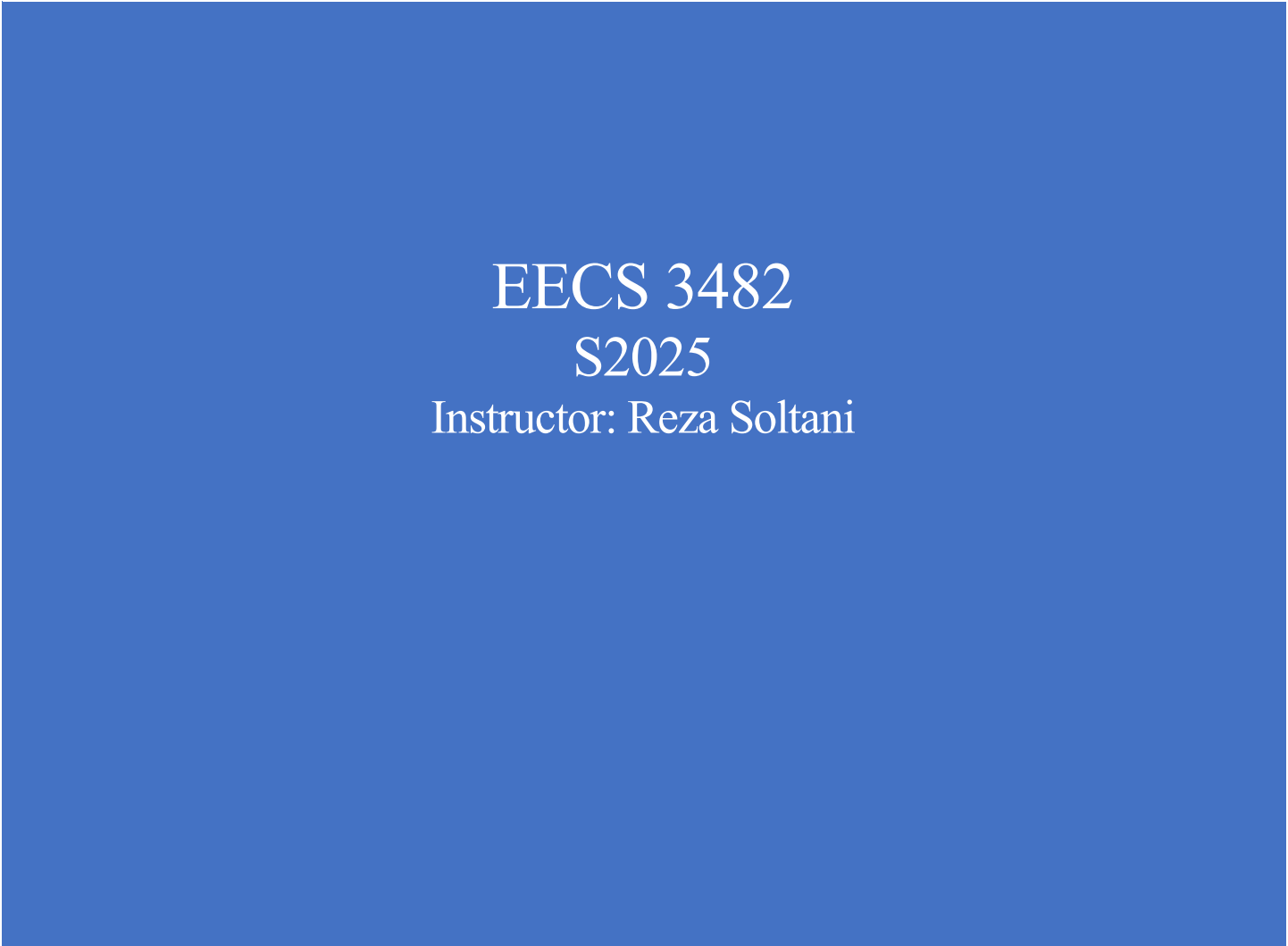
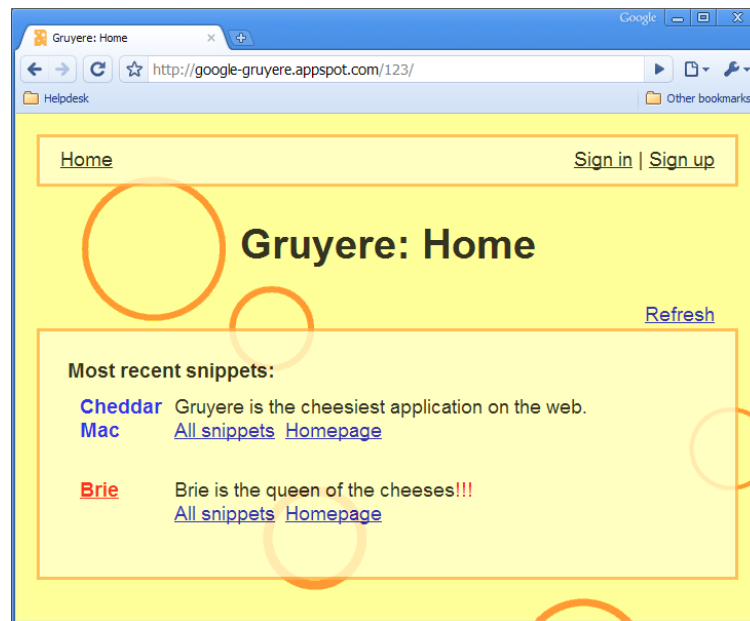




Lab 6 –Web Security



EECS 3482
S2025
Instructor: Reza Soltani



Part 1 - Experiment with the Gruyere platform

The goal of this section is to become familiar with various web attacks and their countermeasures by experimenting with an online platform from Google called [Gruyere CodeLab](#).

Your task (should you accept it!) is to play the role of a hacker and identify and exploit security bugs on the Gruyere Code lab platform.

You **do not need to run Gruyere Code lab locally** on your own machine (or on EECSSVM Labs) to complete this lab. The entire lab can be completed online through Gruyere's website. However, you *do* have the option to download and run the full application locally if you wish.

In this lab, you will learn the following:

- How an application can be attacked using common web security exploits such as cross-site scripting (XSS) and cross-site request forgery (CSRF).
- How to find, fix, and avoid vulnerabilities such as XSS, CSRF, denial-of-service (DoS), information disclosure, and remote code execution.

The code lab is based on a web application called **Gruyere**.

From its website:

Gruyere /gru:'jɛər/ - a small, cheesy web application that allows its users to publish snippets of text and store assorted files. "Unfortunately," Gruyere has multiple security bugs ranging from cross-site scripting and cross-site request forgery, to information disclosure, denial of service, and remote code execution. The goal of this code lab is to guide you through discovering some of these bugs and learning ways to fix them both in Gruyere and in general.

Lab Instructions

1. To become familiar with the Google Code lab, visit: <https://google-gruyere.appspot.com/> and follow the step-by-step tutorials. These tutorials will explain the purpose of the web application, its files and features, and how to interact with it. They will also introduce the various types of vulnerabilities found in the Gruyere application.
2. For the purposes of this lab, you are only required to test and report on **XSS** and **CSRF** vulnerabilities. However, you are encouraged to explore other attack types covered in the Code lab as well.

Your lab report **must include** the following types of attacks:

- **File Upload XSS**
 - **Reflected XSS** (Simple or via AJAX)
 - **Stored XSS** (Simple, HTML, or AJAX)
 - **CSRF**
3. **Code lab does not require any downloads or local installations.** All lab steps can be completed directly on the platform. Each user receives a unique instance ID to interact with the application, so every student will have their own isolated session.
 4. **When you are ready to begin**, access the vulnerable application and start the lab by visiting: <https://google-gruyere.appspot.com/start>
 5. **Record your auto-generated instance ID** in your lab report before entering the site.
 6. Once inside the site, you can view snippets, sign in, or sign up.

7. Familiarize yourself with the application, then follow the instructions in the Code lab tutorial to perform the XSS and CSRF attacks as described.
8. For each attack, take a screenshot with your **Gruyere URL** visible in the **screenshot**. Then document the **attack type**, **steps taken**, the **payload used**, and the **result**.
9. While the website provides suggested countermeasures, you must also propose **one additional possible countermeasure** for each attack.
10. Make sure your report includes your full Gruyere URL with the **instance ID** included.

Part 2 – Thought Experiment

Answer to **ONE** of the following scenarios in one to two paragraphs using your own words (250 words max):

Scenario 1

Biometric login has become the standard, and your company implements retina scans as a secure method of user authentication. At the time, it's praised for being more reliable than passwords and harder to spoof.

Years later, long after you've left the company, a massive data breach exposes the stored retina scan data of millions of users. Unlike passwords, people can't reset or replace their eyes - meaning their biometric identity is now permanently compromised.

Address the following questions in your answer:

- Are you still ethically accountable or professionally responsible for having helped build that system, even though you followed best practices at the time?
- What long-term obligations do developers and engineers have to users when working with immutable data like biometrics?
- Should certain types of data never be used for authentication at all?

Scenario 2

A user installs a well-reviewed health tracking app to monitor sleep, activity, and heart rate. The app promises to protect user privacy and encrypt all data - and it does. The user consents to share their data under clear terms of service, trusting the app to use the data for personalized insights and health recommendations.

Later, the health app forms a partnership with a data analytics firm to improve its features. That firm, in turn, licenses aggregate insights to a marketing agency - and that agency sells access to insurers looking to refine their premium models.

Soon, users who sleep irregular hours or show signs of high stress start receiving higher insurance quotes - not because of a data breach, but because of perfectly legal, contractually permitted data flows.

Everything was technically secure. All parties followed the law. But the user had no idea their trusted health app would indirectly help raise their insurance costs.

Address the following questions in your answer:

- Was trust violated, even if security wasn't?
- Can consent truly be informed when data passes through layers of partnerships and business models the user never sees?
- Should security and privacy engineers have a role in limiting or flagging ethically questionable data usage, even when it's legal?
- How do we design systems that respect user intent, not just legal checkboxes?